Production-Level Enterprise Cybersecurity Project Plans

Project 1: Enterprise LLM & GenAI Security Gateway (AI Security)

Project Information

As enterprises rapidly adopt Generative AI, preventing data leakage and malicious prompt injections is a top industry priority. This project involves building a secure proxy gateway that sits between internal corporate users/applications and external LLM APIs (like OpenAI, Anthropic, or internal models). The gateway intercepts all prompts, scrubs them for Personally Identifiable Information (PII) and Protected Health Information (PHI), blocks prompt injection attacks, and provides full audit logging for compliance.

Tech Stack

- **Language:** Python.
- **Backend Framework:** FastAPI (for high-throughput proxying).
- **AI/Security Tooling:** LangChain, Microsoft Presidio (for NLP-based PII redaction), Rebuff (for prompt injection detection).
- **Database & Caching:** PostgreSQL (for audit logs), Redis (for rate limiting and semantic caching).
- **Deployment:** Docker, Kubernetes.

Expected Impact

Enables the business to safely adopt Generative AI technologies without violating data privacy regulations (GDPR, HIPAA, SOC2) or risking intellectual property leaks. Provides security teams with a centralized control plane for all corporate AI interactions.

4-Week Development Timeline

- **Week 1: Gateway Architecture & Proxy Routing**
    - Day 1-2: Architect the proxy service to intercept outgoing API calls to popular LLM providers.
    - Day 3-5: Build core FastAPI routing, authentication logic (verifying internal user identity), and Redis rate limiting.
    - Day 6-7: Implement standard logging of prompts and responses to PostgreSQL.
- **Week 2: Data Loss Prevention (DLP) & Redaction**
    - Day 1-3: Integrate Microsoft Presidio to analyze outgoing prompts for PII (Credit Cards, SSNs, Names, Internal Project Codes).

    - Day 4-6: Implement anonymization logic (replacing real data with placeholders before sending to the LLM, and deanonymizing the response upon return).
    - Day 7: Write unit tests to ensure no sensitive regex patterns bypass the filter.
- **Week 3: Threat Prevention & Prompt Injection Defense**
    - Day 1-3: Integrate heuristic and ML-based checks (e.g., Rebuff) to detect malicious prompt injections ("ignore previous instructions").
    - Day 4-6: Implement content safety filters to block generated responses that violate corporate policy (e.g., generating malicious code).
    - Day 7: Stress test the latency added by the proxy layer; optimize using async processing.
- **Week 4: Auditing, Dashboarding & Deployment**
    - Day 1-3: Build a compliance dashboard for the Security Operations Center (SOC) to monitor blocked prompts and usage metrics.
    - Day 4-5: Implement Role-Based Access Control (RBAC) defining which departments have access to which AI models.
    - Day 6-7: Containerize the application, configure Kubernetes horizontal pod autoscaling, and finalize documentation.

Project 2: Identity Threat Detection and Response (ITDR) Platform

Project Information

With the perimeter shifting from the network to the user, Identity is the primary attack vector. This platform ingests telemetry from Identity Providers (IdPs) like Azure AD (Entra ID) and Okta. It uses behavioral analytics and graph databases to detect token theft, MFA fatigue attacks, impossible travel, and privilege escalation, immediately mapping the "blast radius" of a compromised account.

Tech Stack

- **Languages:** Python, Go.
- **Data Ingestion:** Apache Kafka or AWS Kinesis.
- **Databases:** Neo4j (Graph Database for mapping identity relationships), Elasticsearch.
- **Integrations:** Microsoft Graph API, Okta API.
- **Analytics:** Pandas, Scikit-learn (for anomaly detection).

Expected Impact

Stops account takeovers and lateral movement before attackers can access critical infrastructure. Shifts the SOC focus from reactive endpoint alerts to proactive identity protection, effectively neutralizing credential-based attacks (which account for the majority of modern breaches).

4-Week Development Timeline

- **Week 1: Identity Telemetry Ingestion**
  - Day 1-3: Establish secure, authenticated connections to Azure AD and Okta via APIs.
  - Day 4-6: Build ingestion pipelines to stream sign-in logs, audit logs, and directory changes into Apache Kafka.
  - Day 7: Normalize the data into a unified identity event schema.
- **Week 2: Graph Database & Blast Radius Mapping**
  - Day 1-3: Deploy Neo4j and model the identity graph (Users, Groups, Roles, Devices, Applications).
  - Day 4-6: Write logic to automatically map the relationships (e.g., User A belongs to "Domain Admins" and has access to "Production AWS").
  - Day 7: Develop queries to instantly calculate the potential impact (blast radius) if a specific user is compromised.
- **Week 3: Behavioral Analytics & Threat Detection**
  - Day 1-3: Implement heuristic rules for known attacks (e.g., brute-force, MFA bombing/fatigue, password spraying).
  - Day 4-6: Develop machine learning models to establish baseline user behavior and flag anomalies (e.g., impossible travel, accessing abnormal resources).
  - Day 7: Route high-fidelity alerts to the Elasticsearch backend.
- **Week 4: Automated Response & SOC UI**
  - Day 1-3: Build a custom API that can issue commands back to the IdP (e.g., force password reset, revoke all active sessions, suspend user).
  - Day 4-5: Develop a SOC dashboard visualizing the identity graph and highlighting active threats.
  - Day 6-7: Conduct red team simulations (simulating token theft) to test the platform's detection and response capabilities.

Project 3: Automated Ransomware Containment & Incident Response Orchestrator

Project Information

A high-speed, automated incident response (IR) orchestrator that operates at machine speed. When a severe threat (like ransomware or hands-on-keyboard activity) is detected by an Endpoint Detection and Response (EDR) tool, this platform instantly triggers an automated playbook: isolating the host from the network, capturing a memory dump for forensics, and suspending the user's active directory account.

Tech Stack

- **Language:** Python.
- **Orchestration Framework:** Cortex XSOAR, Tines, or custom AWS Step Functions.

- **Integrations:** CrowdStrike Falcon API / Microsoft Defender for Endpoint API, Active Directory/Azure AD APIs.
- **Forensics Tools:** KAPE (Kroll Artifact Parser and Extractor), Volatility.
- **Infrastructure:** AWS (Lambda, S3).

Expected Impact

Reduces ransomware dwell time from hours or days to absolute seconds. Automates critical Tier 1 and Tier 2 SOC tasks, preventing lateral ransomware spread across the corporate network during off-hours, and preserves crucial forensic evidence before it is destroyed by the attacker.

4-Week Development Timeline

- **Week 1: API Integration & Alert Ingestion**
  - Day 1-2: Setup the orchestration environment (e.g., AWS Step Functions or a dedicated SOAR instance).
  - Day 3-5: Integrate securely with the EDR API (e.g., CrowdStrike/Defender) via Webhooks to ingest high-severity alerts in real-time.
  - Day 6-7: Parse alert payloads to extract critical indicators (Hostname, IP, User, Process Hash).
- **Week 2: Zero-Trust Containment Playbooks**
  - Day 1-3: Write the automated playbook logic to instantly issue "Network Containment" commands back to the EDR, isolating the machine while maintaining the administrative management connection.
  - Day 4-6: Integrate with the Identity Provider to automatically suspend the compromised user account and revoke session tokens.
  - Day 7: Test the containment workflow in a safe, sandboxed lab environment.
- **Week 3: Automated Forensic Evidence Collection**
  - Day 1-3: Develop scripts utilizing the EDR's live response capabilities to automatically execute KAPE or take a RAM dump upon containment.
  - Day 4-6: Configure secure, automated upload of forensic artifacts to a hardened, write-once-read-many (WORM) AWS S3 bucket for the IR team.
  - Day 7: Establish chain-of-custody logging for all automated actions.
- **Week 4: Ticketing, Communication & Tuning**
  - Day 1-2: Integrate the orchestrator with IT Service Management tools (Jira/ServiceNow) to auto-generate and assign incident tickets with enriched data.
  - Day 3-4: Configure automated Slack/Microsoft Teams alerts for the on-call incident response team.
  - Day 5-7: Perform table-top exercises and live simulations of a ransomware outbreak to measure the platform's response time and finalize documentation.

REQUIRED PROTOCOL FOR ALL ENGINEERING TEAMS

IMPORTANT INSTRUCTION FOR ALL TEAMS: Before commencing development, every team must create their respective GitHub repository. You are required to push commits daily for every week of the project lifecycle. This commit history will serve as the strict metric for your daily productivity tracking. Without consistent daily commits, your evaluation will not be processed by zaalima. Ensure professional commit messages, logical branch management, and adherence to standard collaborative workflows.